TIP In the repository accompanying the book, you will find useful commands for querying the PostgreSQL database directly and verifying schemas and data generated by the application (Chapter05/05-intermediate/catalog-service/README.md).

Manual verification of data persistence is fine, but automated verification is better. That's what the next section is all about.

5.3 Testing data persistence with Spring and Testcontainers

In the previous sections, we added data persistence functionality to an application by developing against a PostgreSQL database in a container, which is the same technology used in production. That was a good step toward the environment parity recommended by the 15-Factor methodology. Keeping all environments as similar as possible improves the quality of the project.

Data sources are one of the primary causes of differences across environments. It's common practice to use an in-memory database while developing locally—something like H2 or HSQL. But that affects the predictability and robustness of your applications. Even if all relational databases speak the SQL language and Spring Data JDBC provides generic abstractions, each vendor has its own dialect and unique features that make it essential to use the same database in production that you use in development and testing. Otherwise, you may not catch errors that might happen only in production.

"What about testing?" you might ask. That is an excellent question. Another reason for using in-memory databases is to make integration tests more accessible. However, integration tests are also supposed to test the integration with your application's external services. Using something like